

```

In [1]: # Import Libraries
import numpy as np          # Numerical operations
import matplotlib.pyplot as plt # Plot graphs
import tensorflow as tf      # Deep Learning framework

from tensorflow.keras.datasets import mnist          # Load MNIST data
from tensorflow.keras.models import Sequential      # Model type
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical    # One-hot encoding

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data() # Load data

# Preprocessing
X_train = X_train.reshape(-1, 28, 28, 1) # Add channel dimension
X_test = X_test.reshape(-1, 28, 28, 1)    # Add channel dimension

X_train = X_train / 255.0 # Normalize pixels
X_test = X_test / 255.0   # Normalize pixels

y_train = to_categorical(y_train, 10) # Convert labels to one-hot
y_test = to_categorical(y_test, 10)   # Convert labels to one-hot

# Build CNN model
model = Sequential() # Initialize model

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))) # Conv Layer
model.add(MaxPooling2D(2,2)) # Pooling Layer

model.add(Conv2D(64, (3,3), activation='relu')) # Conv Layer
model.add(MaxPooling2D(2,2)) # Pooling Layer

model.add(Flatten()) # Convert to 1D

model.add(Dense(128, activation='relu')) # Fully connected layer
model.add(Dropout(0.5)) # Reduce overfitting

model.add(Dense(10, activation='softmax')) # Output Layer

# Compile model
model.compile(optimizer='adam', # Optimizer
              loss='categorical_crossentropy', # Loss function
              metrics=['accuracy']) # Evaluation metric

# Train model
history = model.fit(X_train, y_train, # Training data
                   epochs=5, # Number of iterations
                   batch_size=64, # Batch size
                   validation_split=0.1) # Validation data

```

```

test_loss, test_acc = model.evaluate(X_test, y_test) # Test performance
print("Test Accuracy:", test_acc) # Print accuracy

# Plot accuracy
plt.plot(history.history['accuracy']) # Train accuracy
plt.plot(history.history['val_accuracy']) # Validation accuracy
plt.legend(['Train', 'Validation']) # Legend
plt.title("Accuracy") # Title
plt.show() # Show plot

# Prediction
import random # Random selection

index = random.randint(0, len(X_test)-1) # Pick random image
sample = X_test[index] # Get image

plt.imshow(sample.reshape(28,28), cmap='gray') # Show image
plt.show() # Display image

prediction = model.predict(sample.reshape(1,28,28,1)) # Predict
print("Predicted:", np.argmax(prediction)) # Show result

```

C:\Users\aida-lab\AppData\Local\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/5

844/844 ————— **17s** 17ms/step - accuracy: 0.9190 - loss: 0.2637 - val_accuracy: 0.9840 - val_loss: 0.0534

Epoch 2/5

844/844 ————— **14s** 16ms/step - accuracy: 0.9712 - loss: 0.0961 - val_accuracy: 0.9848 - val_loss: 0.0492

Epoch 3/5

844/844 ————— **21s** 17ms/step - accuracy: 0.9788 - loss: 0.0711 - val_accuracy: 0.9887 - val_loss: 0.0389

Epoch 4/5

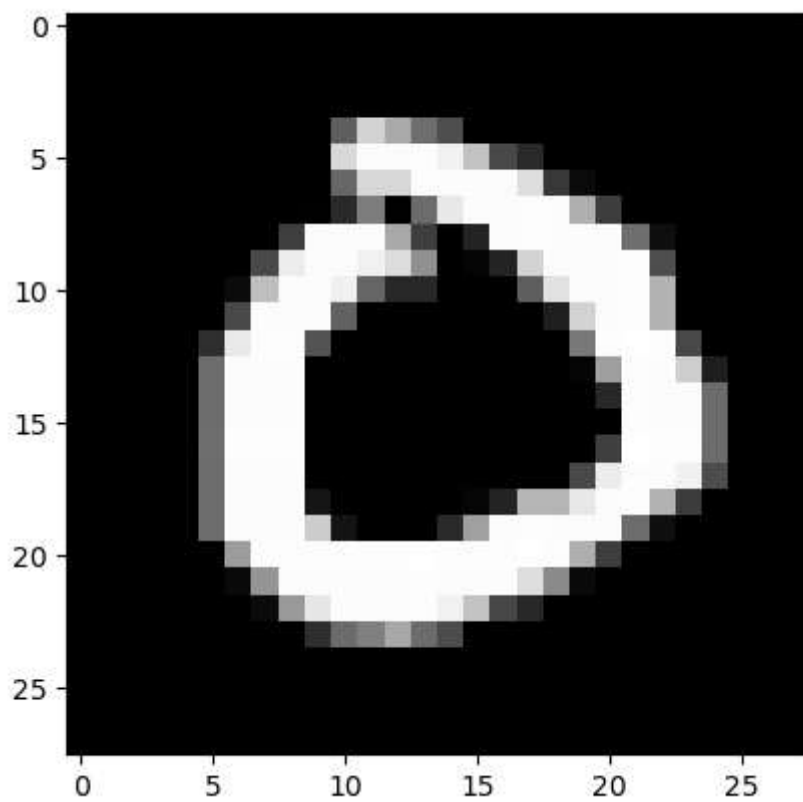
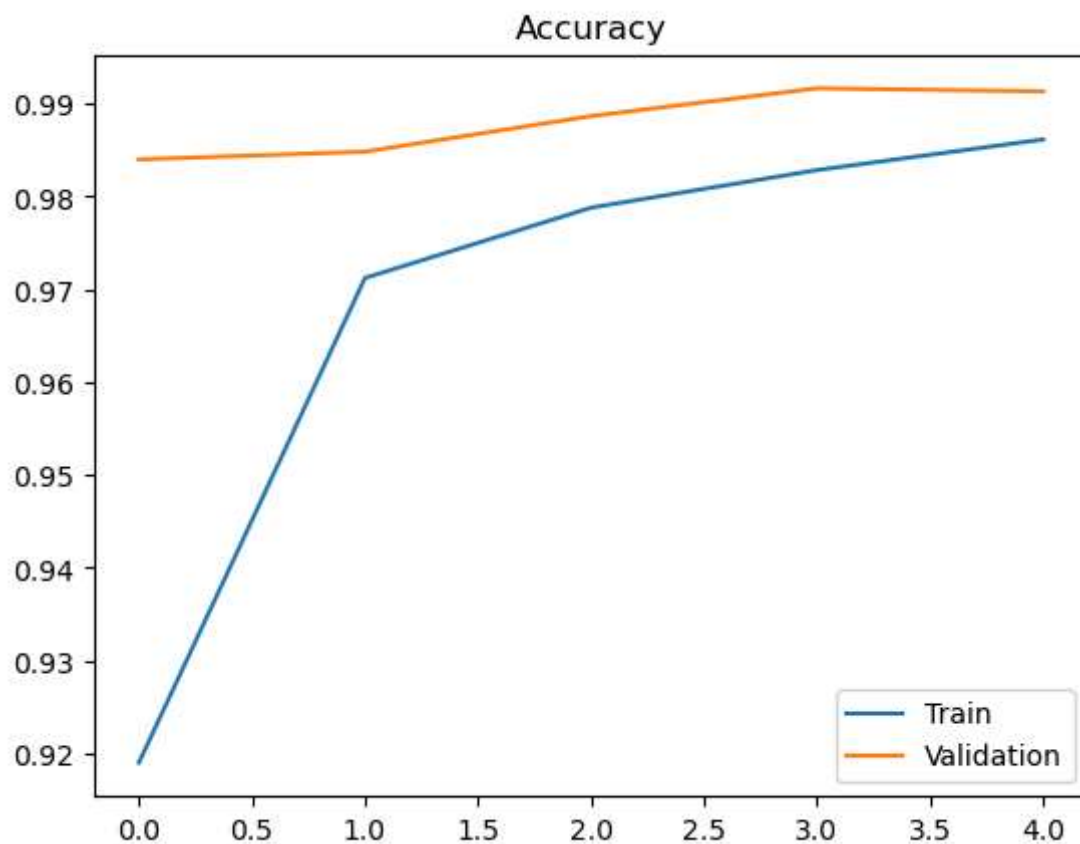
844/844 ————— **20s** 16ms/step - accuracy: 0.9829 - loss: 0.0575 - val_accuracy: 0.9917 - val_loss: 0.0321

Epoch 5/5

844/844 ————— **21s** 16ms/step - accuracy: 0.9861 - loss: 0.0464 - val_accuracy: 0.9913 - val_loss: 0.0314

313/313 ————— **1s** 4ms/step - accuracy: 0.9910 - loss: 0.0256

Test Accuracy: 0.990999996621399



1/1 — 0s 147ms/step
Predicted: 0